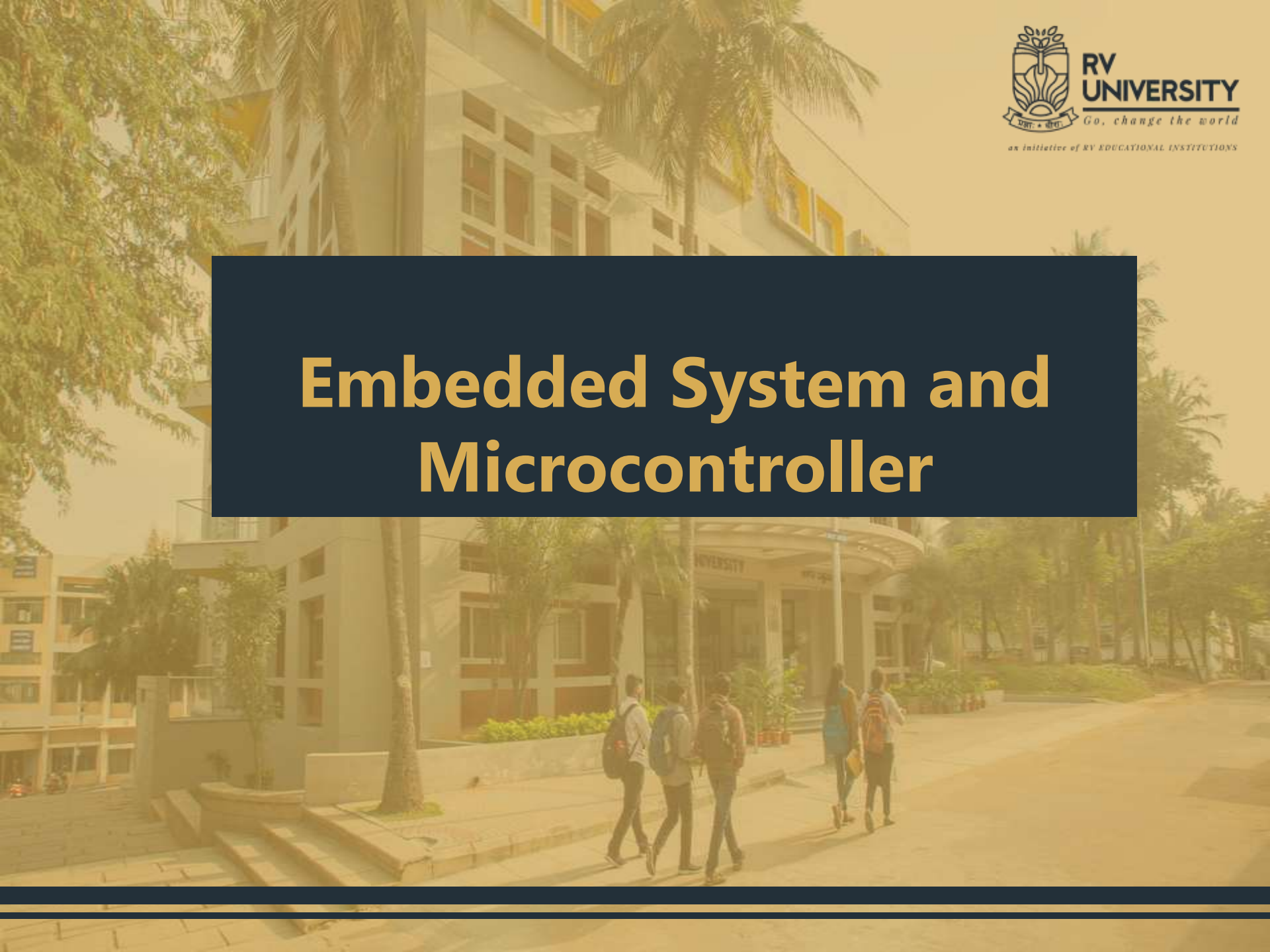


Embedded System and Microcontroller

Microprocessors & Microcontrollers

: Arm Cortex M0+

(Using RP2040)

Overview of ARM architectures and processors: Cortex-A, Cortex-R, and Cortex-M families

Cortex-M Processor Families

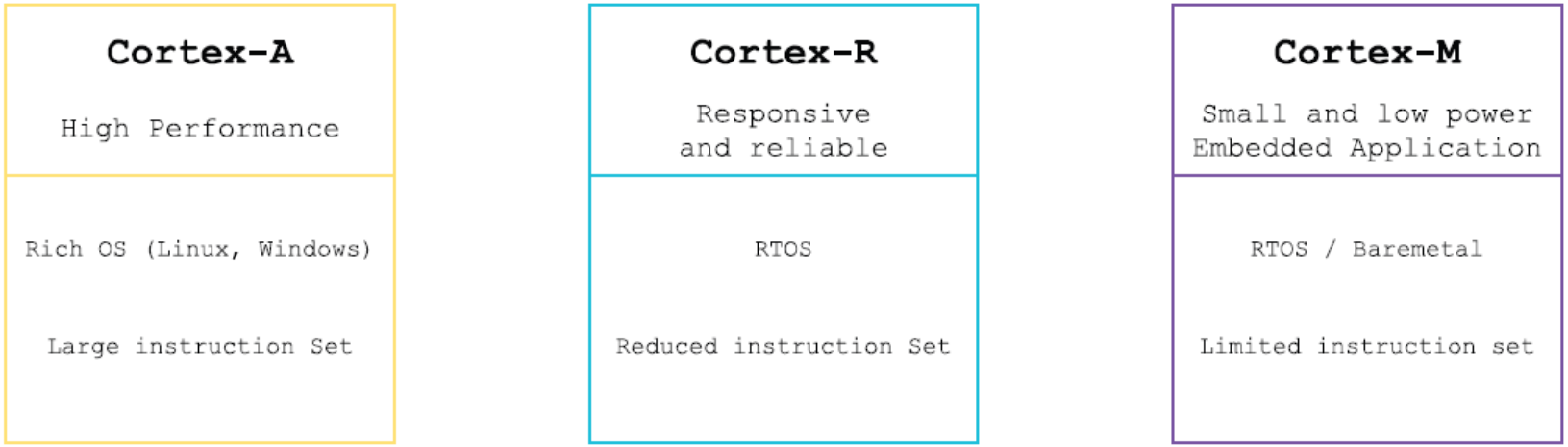


Figure 3.1 Cortex families for system-on-chip design

Figure 3.1 shows the processors in the Cortex-M family are the Arm cores that have been optimized for size and low power consumption.

Cortex-M Flavors

- **Cortex-M Family Classification by Architecture**
- **Arm-v6-M Architecture**
  - **Focus**: Low power consumption.
  - **Instruction Set**: Limited to 16-bit Thumb instructions for optimized application size.
  - **Performance**: Lower benchmark results due to minimal instruction set and features.
- **Arm-v7-M Architecture**
  - **Variants**:
    - **Standard Arm-v7-M**: Core features.
    - **Arm-v7E-M**: Enhanced version with DSP (Digital Signal Processing) extensions.

Cortex-M Flavors

- **Arm-v8-M Architecture**
- **Baseline Version**:
  - Subset of the full architecture.
  - Fewer gates and simpler instruction set.
  - Enhanced security and performance compared to Arm-v6-M.
- **Mainline Version**: Full feature set of Arm-v8-M.
- **Arm-v8.1-M Architecture**
- **Key Addition**: Helium extension.
- **Benefits**: Enables vector instructions and boosts performance.

Cortex-M Flavors

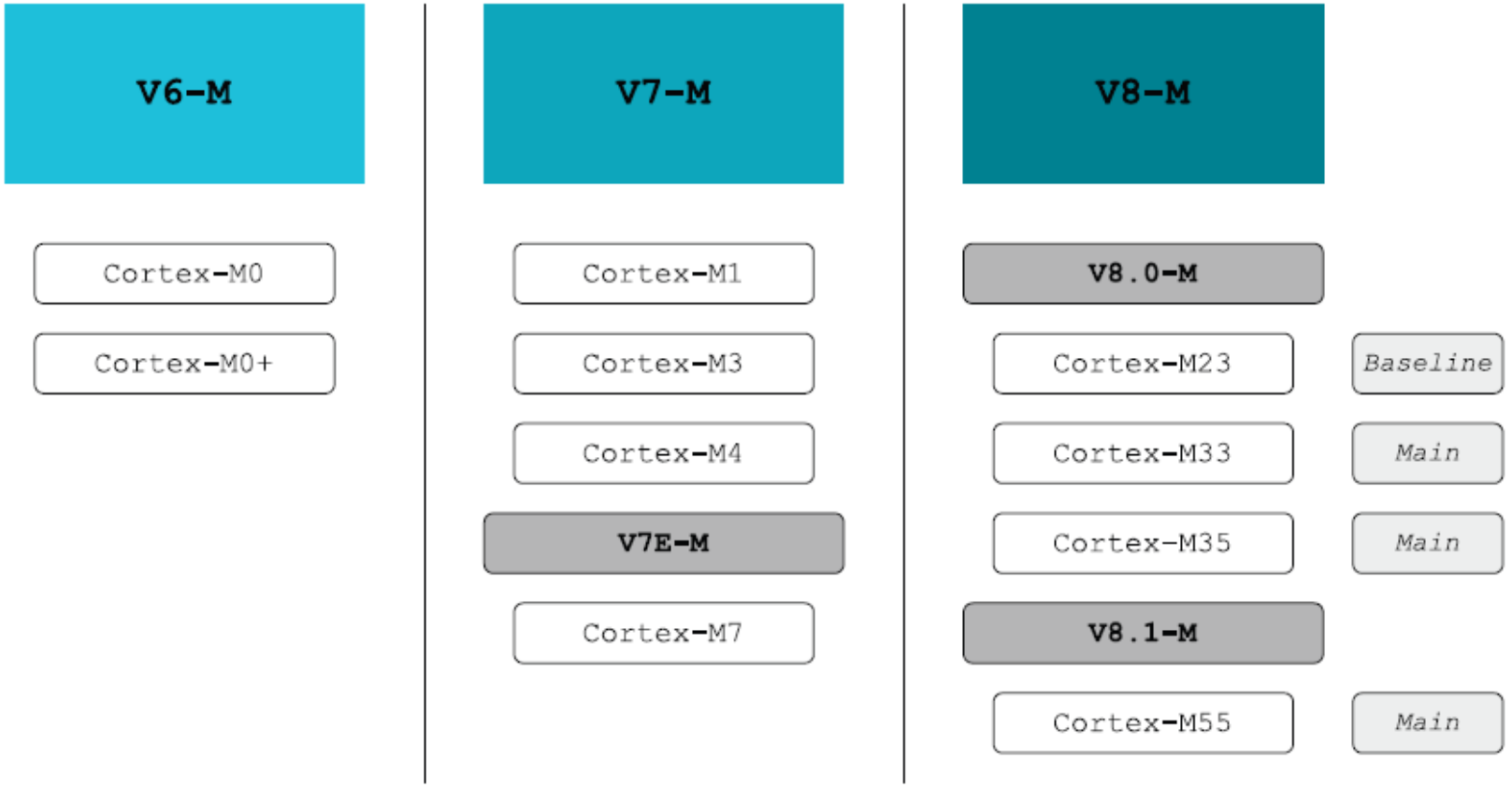


Figure 3.2 Cortex-M family classified according to architecture

Arm Cortex-M Programmer's Model

- **Programmer's Model Overview**
- Represents all computer elements affected by instruction execution.
- Includes state variables and registers for instruction operations.
- **Instruction Set Architecture**
- **ARM Instruction Sets**:
  - Originally 32-bit coding for performance but memory inefficient.
  - Introduction of **Thumb** (16-bit instruction set): Smaller size but reduced capability.
  - **Thumb-2**: Combines 16- and 32-bit instructions for better efficiency and flexibility.

Arm Cortex-M Programmer's Model

- **CISC (Complex Instruction Set Computing)**:
  - Fewer instructions per program (multi-operation instructions).
  - Direct memory operations supported.
  - Varied instruction lengths (1–16 bytes).
  - Hardware complexity and limited clock frequency (up to tens of MHz).

- **RISC (Reduced Instruction Set Computing)**:
  - Simpler instructions with uniform sizes.
  - Operations only on internal registers; memory accessed via LOAD/STORE.
  - Separate units for data transfer and manipulation for efficient pipelining.
  - Simple addressing modes:
    - Access to internal registers.
    - Immediate value load.
    - PC-based access or register indexing.

Arm Cortex-M Programmer's Model

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Cortex-M Instruction Set Features

- **16-bit Thumb Instruction Set**:
  - Smaller code footprint.
  - Conditional instructions (e.g., IT - If-Then).
  - Compare and Branch if Zero/Non-Zero (CBZ/CBNZ).
- **32-bit ARM Instruction Set**:
  - Exception handling.
  - Coprocessor access.
  - DSP/media instructions.
  - Improved performance.

Arm Cortex-M Programmer's Model

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Instruction Set Variants

- **T32**:

  Thumb instruction set for Cortex-M, Cortex-A, and Cortex-R families.

  Universally available across all Cortex families.

- **A32**:

  Used in Cortex-A and Cortex-R; previously called ARM instruction set.

- **A64**:

  64-bit data support; uses 32-bit encoding (for AArch64 state).

- **Advanced SIMD (Single Instruction, Multiple Data)**:

  Adds SIMD arithmetic extensions (32x64-bit or 32x128-bit registers).

- **Floating Point Extensions**:

  Optional extension on Arm-v7 and later.

Processor Modes

1. Execution Privilege Levels

- Privilege levels determine access rights and execution capabilities.

2. Processor Modes

- **Thread Mode**:

  Normal mode for executing programs.

- **Handler Mode**:

  Exception-handling mode.

  Executes with privileged software access.

3. Arm Architecture Support

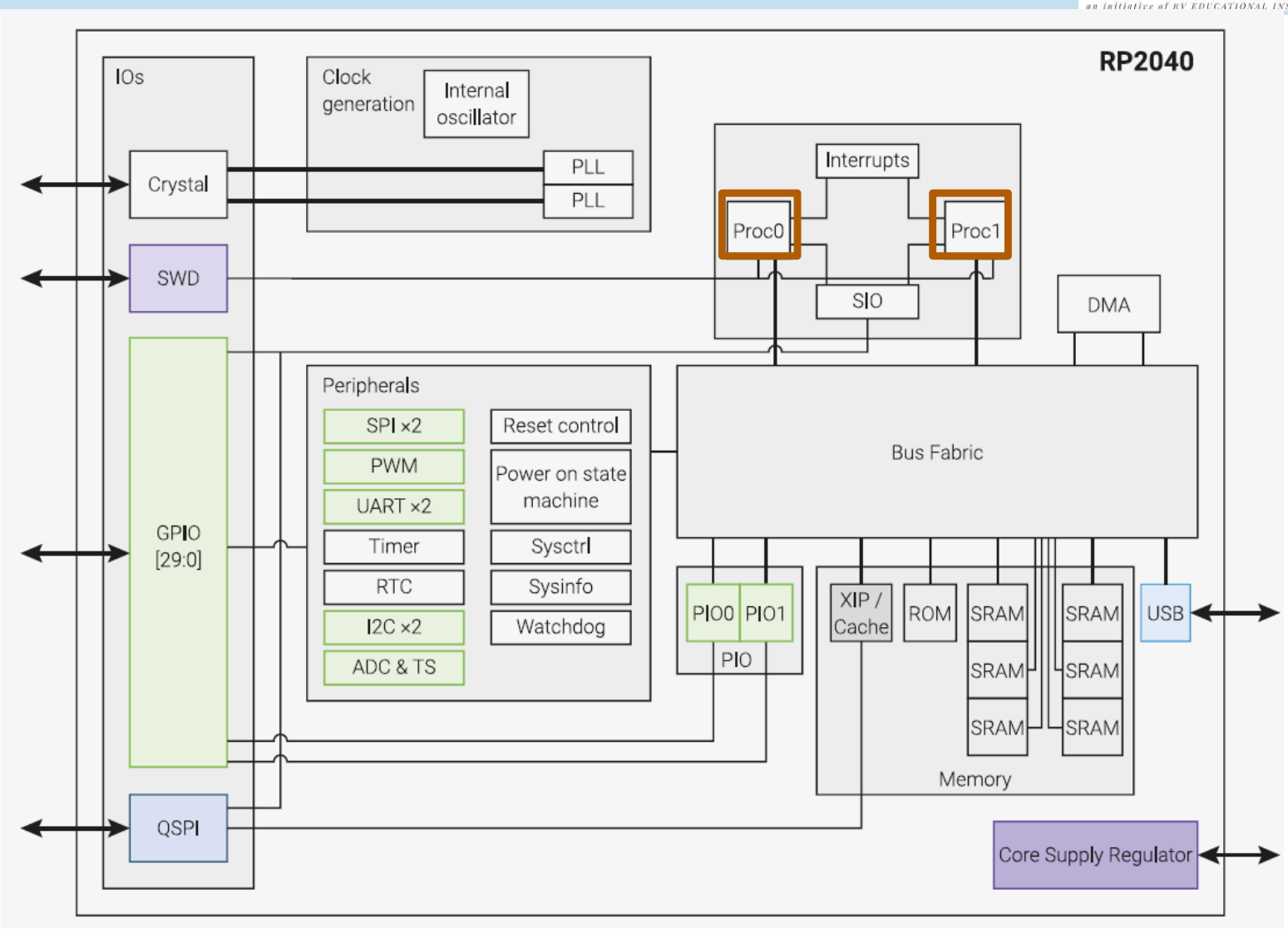
- **Arm-v6-M and Arm-v7-M**:

  Support **Thread Mode** and **Handler Mode**.

- **Arm-v8-M (with TrustZone extension)**:

  Supports **four processing modes** for enhanced security.

Arm-Cortex-M0+ Cores



Arm Cortex-M0+ (ARMv6-M Profile)

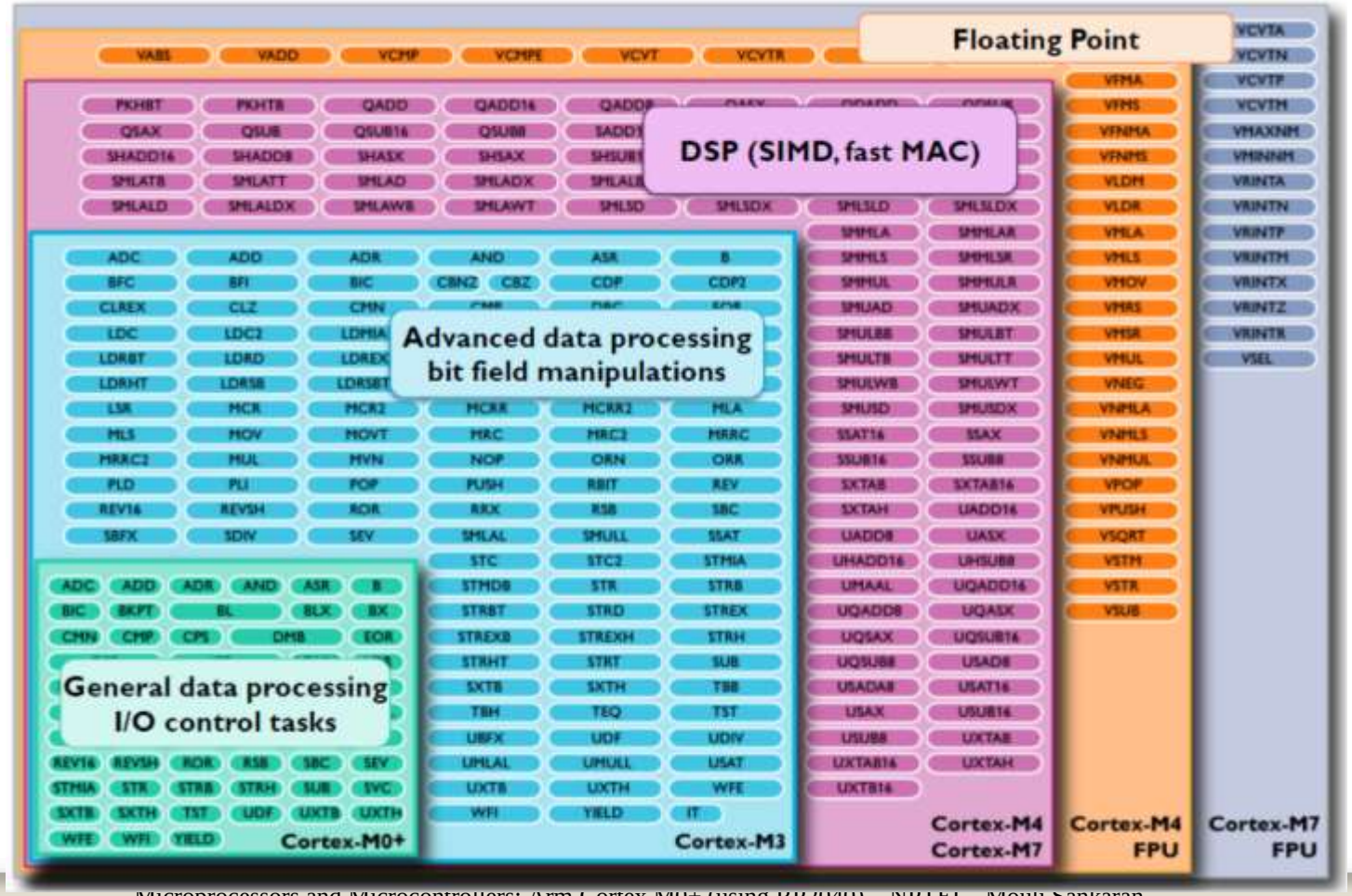
Processor	Arm Architecture	Core Architecture	Thumb®	Thumb®-2	Hardware Multiply	Hardware Divide	Saturated Math	DSP Extensions	Floating Point
Cortex-M0	Armv6-M	Von Neumann	Most	Subset	1 or 32 cycle	No	No	No	No
Cortex-M0+	Armv6-M	Von Neumann	Most	Subset	1 or 32 cycle	No	No	No	No
Cortex-M1	Armv6-M	Von Neumann	Most	Subset	3 or 33 cycle	No	No	No	No
Cortex-M3	Armv7-M	Harvard	Entire	Entire	1 cycle	Yes	Yes	No	No
Cortex-M4	Armv7E-M	Harvard	Entire	Entire	1 cycle	Yes	Yes	Yes	Optional

ARMv7-M: The microcontroller profile for systems supporting only the Thumb instruction set, and where overall size and deterministic operation for an implementation are more important than absolute performance.

ARMv6-M is a **subset** of **ARMv7-M**

Cortex-M Instruction Set

- ARMv6-M supports the Thumb instruction set, including a small number of 32-bit instructions.
- Hence widely used in mobile devices, such as smartphones and tablets



Cortex-M Instruction Set

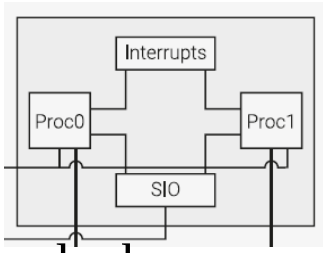
- The Thumb instruction set is a subset of the ARM instruction set architecture developed by ARM Holdings. It is designed to improve code density by using 16-bit instructions instead of the regular 32-bit ARM instructions. The Thumb instruction set allows for more efficient use of memory, which is particularly important in embedded systems with limited resources.
- Key features of the Thumb instruction set include:
- **16-bit Instructions:** Thumb instructions are half the size of regular ARM instructions (32 bits). This reduction in size results in more compact code, which is beneficial for systems with limited memory.
- **Code Density:** By using 16-bit instructions, the Thumb instruction set increases the code density, meaning that more instructions can fit into a given amount of memory.
- **Backward Compatibility:** Processors that support the Thumb instruction set are typically backward compatible with the regular ARM instruction set. This allows for a mix of Thumb and ARM instructions in the same program.

Cortex-M Instruction Set

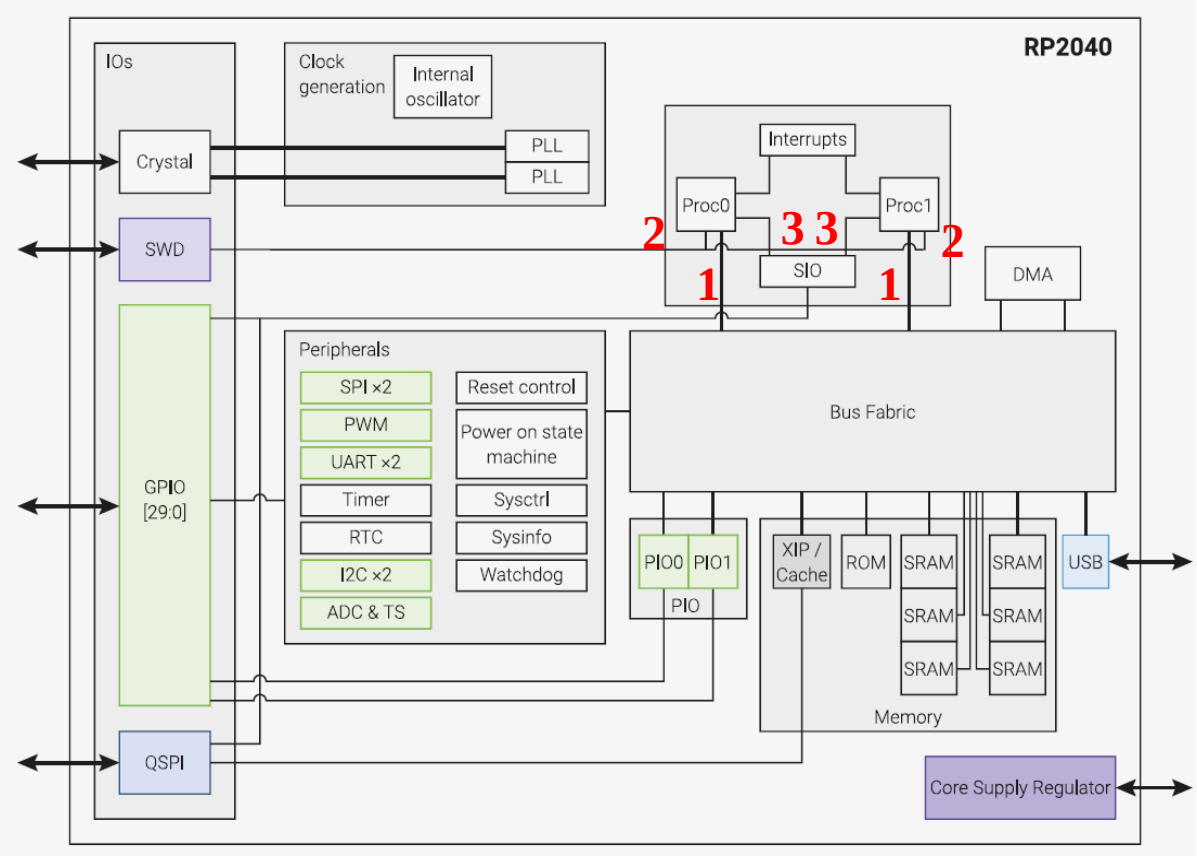
- **Reduced Power Consumption:** The smaller instructions also contribute to reduced power consumption since fetching and decoding smaller instructions require less energy.
- The Thumb instruction set has been widely adopted in ARM Cortex-M microcontrollers, which are commonly used in embedded systems and microcontroller applications. It allows these systems to achieve better performance and efficiency in terms of code size and power consumption.
- It's important to note that there are different versions of the Thumb instruction set, such as Thumb, Thumb-2, and Thumb-EE (Enhanced Efficiency). Thumb-2, for example, extends the Thumb instruction set with additional 32-bit instructions to provide a balance between code density and performance. The choice of Thumb or regular ARM instruction set depends on the specific requirements of the target application.

Arm-Cortex-M0+: Features and Benefits

- Tight integration of system peripherals reduces area and development costs.
- Thumb instruction set combines high code density with 32-bit performance.
- Support for single-cycle I/O access.
- Power control optimization of system components.
- Integrated sleep modes for low-power consumption.
- Fast code execution enables running the processor with a slower clock or increasing sleep mode time.
- Optimized code fetching for reduced flash and ROM power consumption.
- Hardware multiplier.
- Deterministic instruction cycle timing.
- Deterministic, high-performance interrupt handling for time-critical applications.
- Support for system level debug authentication.
- Serial Wire Debug (SWD) reduces the number of pins (5 wires) required for debugging.



Arm-Cortex-M0+: External Interfaces



- The **three interfaces** provided in the processor for **external accesses** are:
  1. External AHB-Lite interface to bus fabric
  2. Debug Access Port (DAP)
  3. Single-cycle I/O Port to SIO peripherals

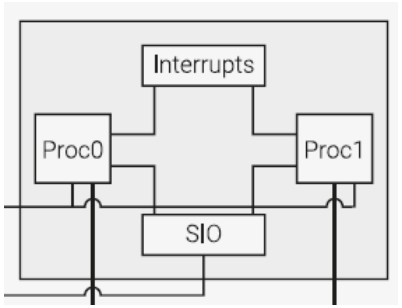
Arm-Cortex-M0+: External Interfaces

- The AHB-Lite (Advanced High-Performance Bus Lite) is a simplified version of the ARM AMBA (Advanced Microcontroller Bus Architecture) AHB (Advanced High-Performance Bus) interface. AHB is a widely used on-chip bus protocol in ARM-based systems for connecting various components such as processors, memory, and peripherals.
- When discussing an "External AHB-Lite interface to bus fabric," it typically means connecting an AHB-Lite interface to the bus fabric, which is the interconnection network that allows different components on a chip to communicate with each other.
- Integrating a DAP with an AHB-Lite interface is common in embedded systems to enable debugging capabilities. The DAP can be connected to the AHB-Lite bus to access and control various components for debugging, such as reading and writing to registers, halting the processor, and inspecting memory.
- A "single-cycle I/O port to SIO (Serial Input/Output) peripherals" suggests a design feature in a digital system where the I/O port can communicate with SIO peripherals in a single clock cycle.

Arm-Cortex-M0+: External Interfaces

- **I/O Port:**
  - An I/O port, or Input/Output port, is a hardware interface used for communication between a digital system and external devices. It allows data to be transferred into or out of the system.
- **Single-Cycle:**
  - "Single-cycle" indicates that the data transfer between the I/O port and the SIO peripherals can be completed within a single clock cycle. This design choice is often made to achieve high-speed communication.
- **SIO Peripherals:**
  - SIO refers to Serial Input/Output peripherals. These peripherals typically communicate using serial protocols like UART (Universal Asynchronous Receiver/Transmitter), SPI (Serial Peripheral Interface), I2C (Inter-Integrated Circuit), or other serial communication standards.

Arm-Cortex-M0+: Configurations



Each processor is configured with the following features:

- Architectural clock gating (for power saving)
- Little Endian bus access (code access is always Little endian)
- Data access is configurable (It is fixed as Little in RP2040)
- Four Breakpoints
- Debug support (via 2-wire debug pins SWD/SWCLK)
- 32-bit instruction fetch (to match 32-bit data bus)
- IOPORT (for low latency access to local peripherals (SIO))
- 26 interrupts
- 8 MPU regions
- All registers reset on power up

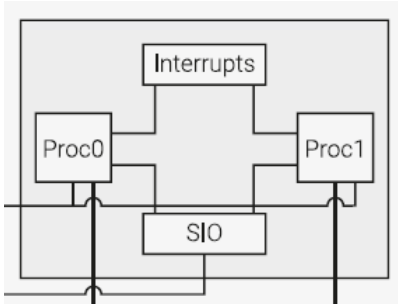
MPU: Memory Protection Unit

Arm-Cortex-M0+: Configurations … contd.

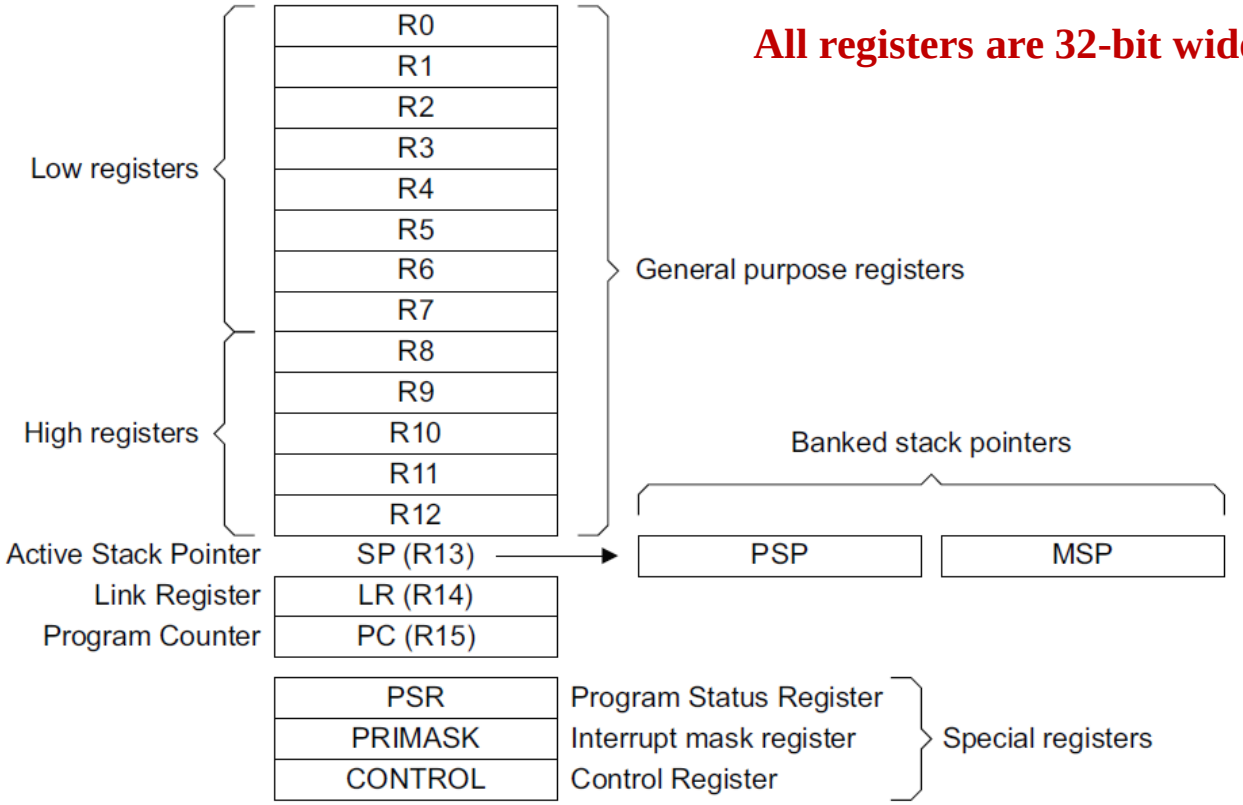
Each processor is configured with the following features:

- Fast multiplier (MULS 32x32 single cycle)
- SysTick timer
- Vector Table Offset Register (VTOR)
- 34 WIC (Wake-up Interrupt Controller) lines (32 IRQ and NMI)
- DAP feature: Halt event support
- DAP feature: SerialWire debug interface (protocol 2 with multidrop support)
- DAP feature: Micro Trace Buffer (MTB) is not implemented

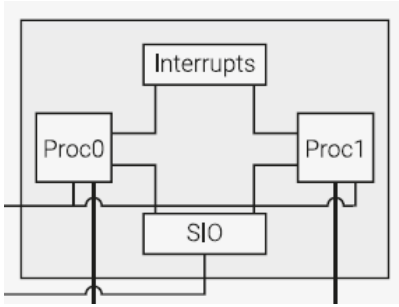
Each M0+ core has its own interrupt controller
which can individually mask out interrupt sources
as required. The same interrupts are routed to both M0+ cores.



Arm-Cortex-M0+: Register Set



All registers are 32-bit wide



PSP: Process Stack Pointer

MSP: Main Stack Pointer (default on reset)

Ref:Ref4_ARM-Cortex-M0+ Devices Generic User Guide, Section 2.1.3 Core Registers

Arm-Cortex-M0+: Register Set

General-purpose registers

R0-R12 are 32-bit general-purpose registers for data operations.

Stack Pointer

The stack pointer (SP) is register R13.

The processor uses a full descending stack, meaning the Stack Pointer holds the address of the last stacked item in memory. When the processor pushes a new item onto the stack, it decrements the Stack Pointer and then writes the item to the new memory location.

Link Register

The Link Register (LR) is register R14. It stores the return information for subroutines, function calls, and exceptions. On reset, the processor sets the LR value to 0xFFFFFFFF.

Program Counter

The Program Counter (PC) is register R15. It contains the current program address.

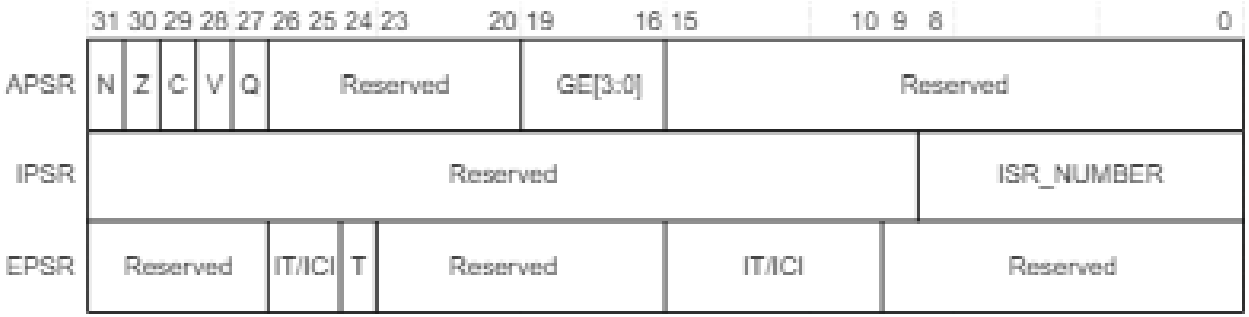
On reset, the processor loads the PC with the value of the reset vector defined in the vector table.

Arm-Cortex-M0+: Register Set

Combined Program Status Register

The Combined Program Status Register (xPSR) consists of the Application Program Status Register (APSR), Interrupt Program Status Register (IPSR), and Execution Program Status Register (EPSR).

These registers are mutually exclusive bit fields in the 32-bit PSR. The bit assignments are as follows:



Arm-Cortex-M0+: Register Set

Priority Mask Register

The PRIMASK register is intended to disable interrupts by preventing activation of all exceptions with configurable priority in the current Security state.

CONTROL register

The CONTROL register controls the stack that is used, the privilege level for software execution when the core is in Thread mode and indicates whether the FPU state is active.

ARM Instruction Format

- Most ARM Cortex-M0+ binary instructions are 16 bits long. There are six 32-bit-long instructions.
- Newer A-series CPUs typically have 32-bit instructions, but if you want to save memory, there is a “thumb” mode. When you switch to “thumb” mode, most of the instructions are 16 bits in size, thus using half the memory.
- The M-series CPUs are designed for embedded processors running with minimal memory.
- let’s consider the format for an ADD instruction. The following is the format of the instruction and what the bits specify:
- Let’s look at each of these fields:
- Opcode: Which instruction are we performing, like ADD or SUB
- Rm and Rn: The two registers to add
- Rd: The destination register, where to put the result of the addition

15-9	8-6	5-3	2-0
OpCode	Rm	Rn	Rd

ARM Instruction Format

Example : ADD R5,R3, R2

- ARM Instruction
- OpCode = 0001100 meaning ADD
- Rm = 010 = 2 (i.e., R2)
- Rn = 011 = 3 (i.e., R3)
- Rd = 101 = 5 (i.e., R5)on Format

Instructions

- Addition in Assembly
  - Example: `ADD r0,r1,r2` (in ARM)

  Equivalent to: `a = b + c` (in C)

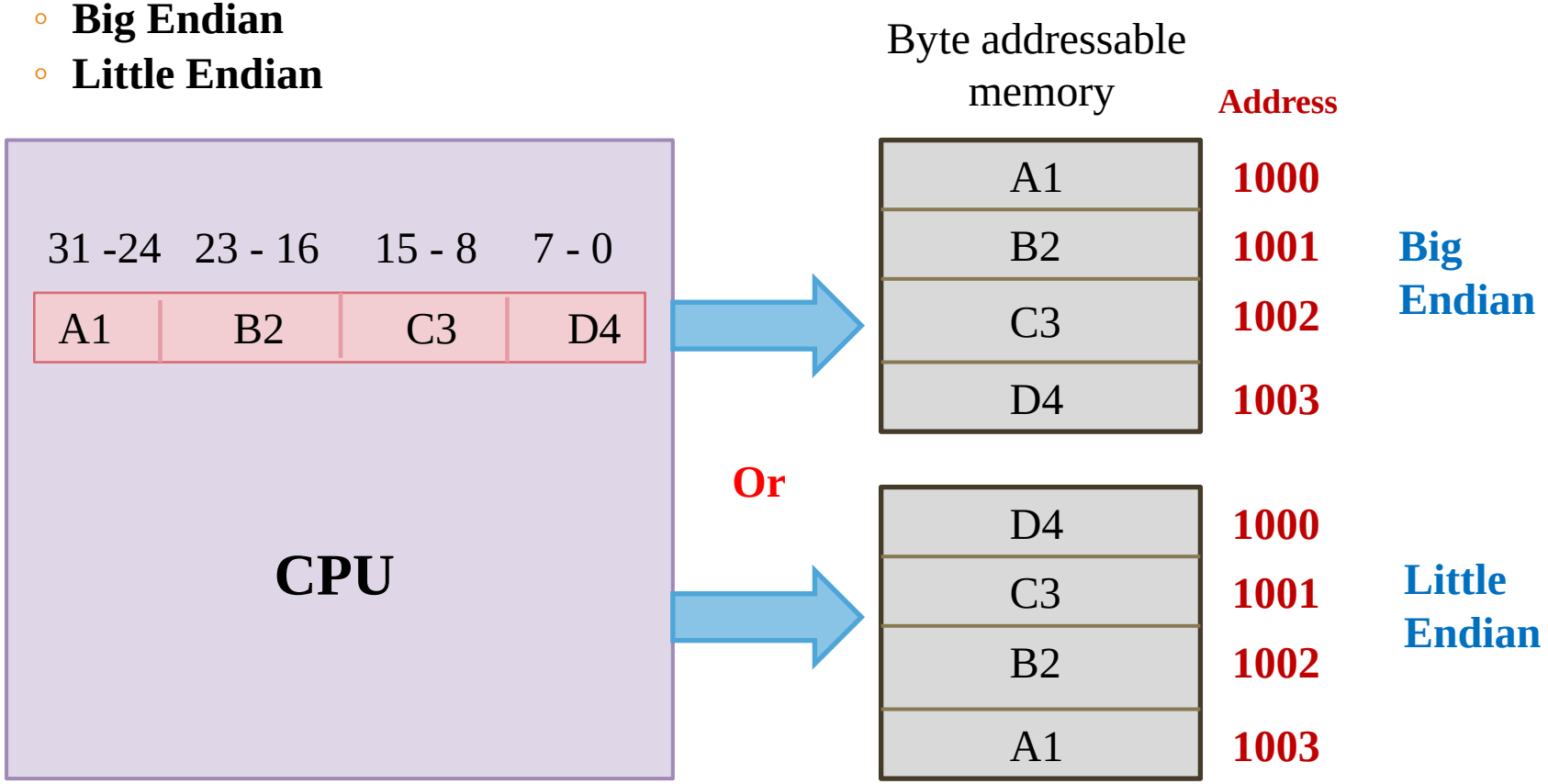
  where ARM registers `r0,r1,r2` are associated with C variables `a, b, c`
- Subtraction in Assembly
  - Example: `SUB r3, r4, r5` (in ARM)

  Equivalent to: `d = e - f` (in C)

  where ARM registers `r3,r4,r5` are associated with C variables `d, e, f`

Big-Endian vs Little-Endian

- When a register content from a processor is moved to memory, it can be saved in two different ways
  - **Big Endian**
  - **Little Endian**



Quiz 1

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Choose the correct option:

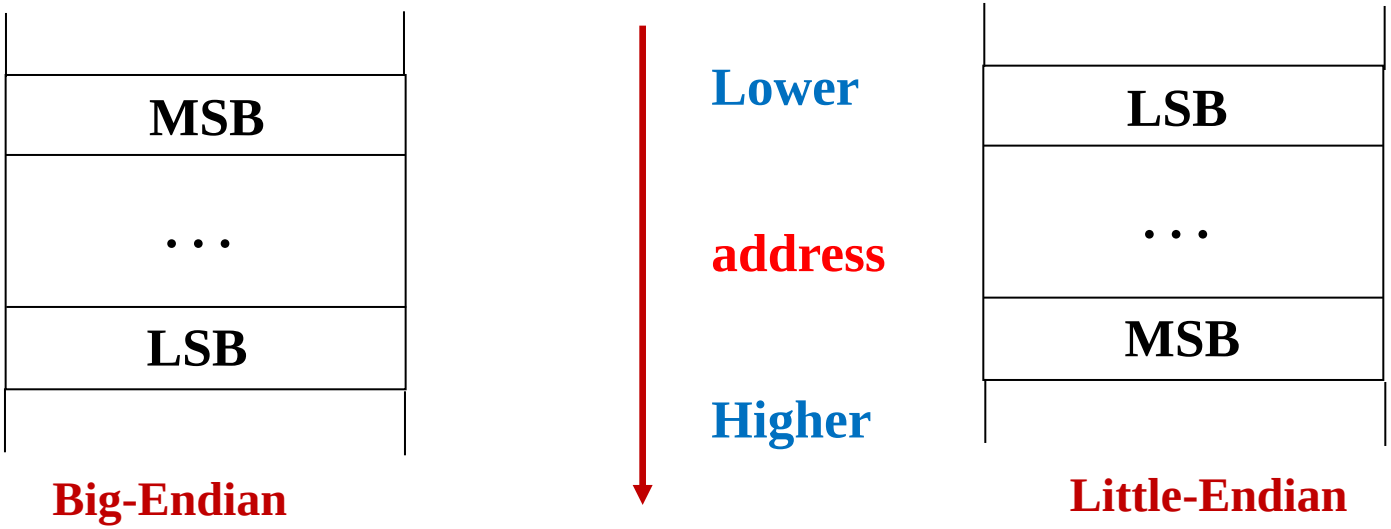
1. A CPU wrote a register content into memory in a Big-Endian mode. When the same content is read back from the memory into a register, the CPU is reading it in Little-Endian mode.

a) The new content in the register will be different from what was written.
b) The content will be the same.
c) The content may be same or different.
d) Endianness need not be the same while writing and reading the contents into/from the memory.

Correct option: a

Note: The endianness needs to be the same. The endianness used while writing a content into the memory **should** be used while reading the same content back into a register.

Big-Endian and Little-Endian



Motorola 68xx, 680x0

IBM Mainframe

HP PA-RISC

Internet TCP/IP

Bi-Endian

ARM

Motorola Power PC

Sun SPARC

MIPS

Intel x86

AMD Opteron

DEC VAX

PIC Microcontroller

Big-Endian and Little-Endian

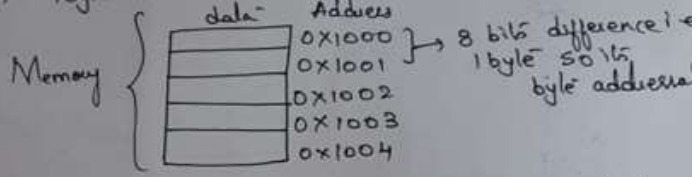
RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

① Write the Each program line output, and store the data to memory, which is of Big Endian.

```
MOV  R1, #0X85
MOV  R2, #0X35
Add  R4, R1, R2
STR  R4, [R3]
```

R3 = 0X1000

Case i) Byte Addressable



Solu:- MOV R1, #0X85 ; Moves the immediate number to GPR R1 ∴ R1 = 0X85 (hexadecimal)

MOV R2, #0X35 ; ⇒ R2 = 0X35

Add R4, R1, R2 ; ⇒ R4 = R1 + R2

= 0X25 + 0X35 Hexadecimal Add

∴ = 0X5A

⇒ R4 = 0X5A

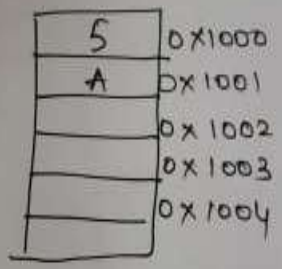
STR R4, [R3] ; Store instruction loads the R4 Register data to memory location pointed by R3 i.e [1000]

∴ Its given memory in Big Endian (BE)

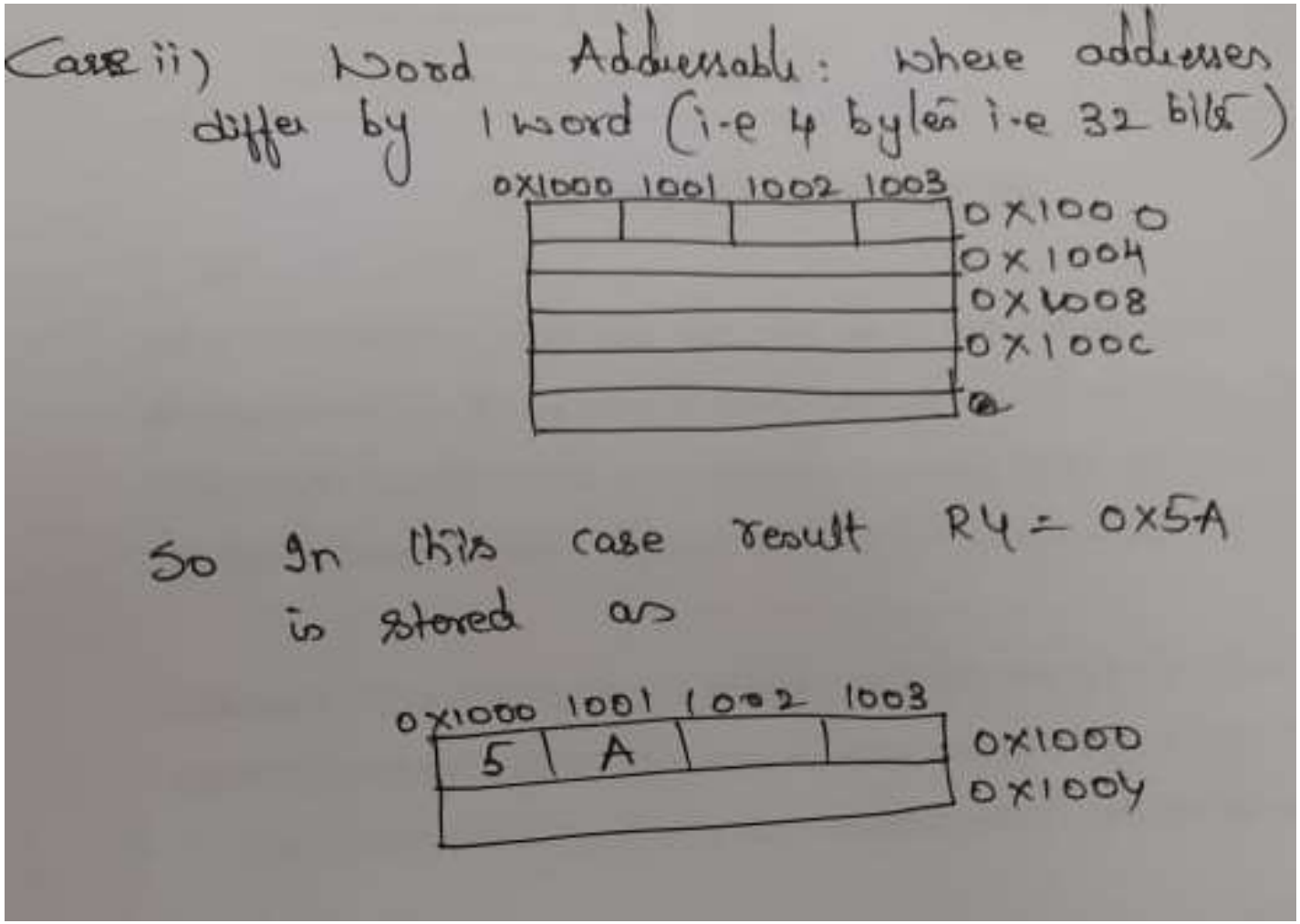
In BE lower byte in Higher Address & Higher byte in lower address.

R4 = 0X5A (Higher Byte → 5, lower byte → A)

∴



Big-Endian and Little-Endian



Endianness Comparison
Advantages and Disadvantages

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Big-Endian

- Easier to determine a sign of the number
- Easier to compare two numbers
- Easier to divide two numbers
- Easier to print

Little-Endian

- Easier for addition and multiplication of multi-precision numbers
- It's easy to read the value in a variety of different datatype sizes
- It's easy to cast the value to a smaller type, for example from **int16_t** to **int8_t** since **int8_t** is the byte at the beginning of **int16_t**

QUiZ

1. **What is the min. number of assembly instructions needed to perform the following**

   a = b + c + d - e;

A. Single instruction
B. Two instructions
C. Three instructions
D. Four instructions

2. **f = (g + h) - (i + j);**

QUiZ

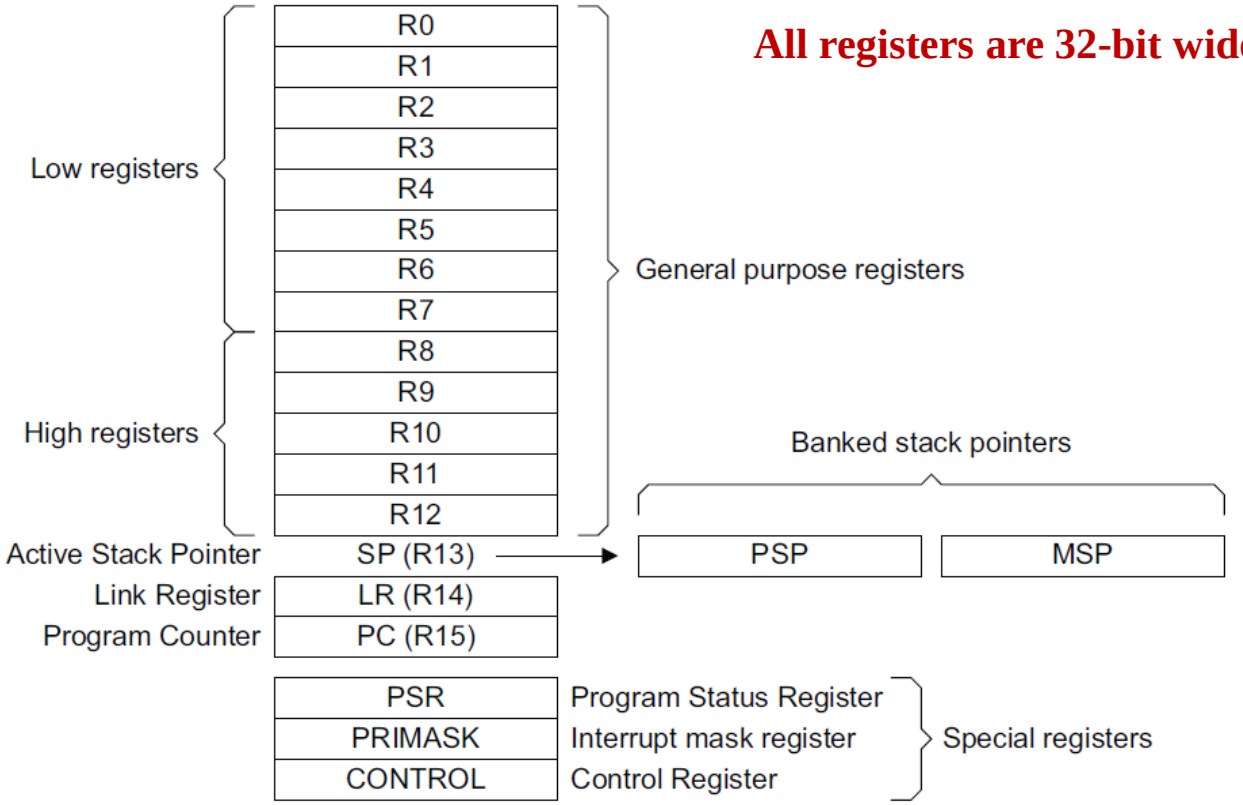
1 solution: Break into multiple instructions

```
ADD r0, r1, r2 ; a = b + c
ADD r0, r0, r3 ; a = a + d
SUB r0, r0, r4 ; a = a - e
```

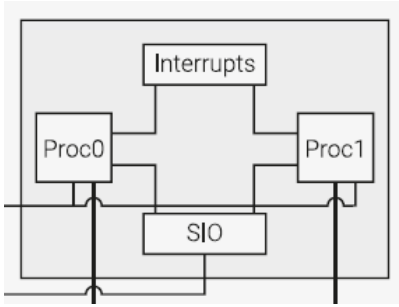
2 solution:

```
ADD r0,r1,r2 ; f = g + h
ADD r5,r3,r4 ; temp = i + j
SUB r0,r0,r5 ; f =(g+h)-(i+j
```

Arm-Cortex-M0+: Register Set



All registers are 32-bit wide



PSP: Process Stack Pointer

MSP: Main Stack Pointer (default on reset)

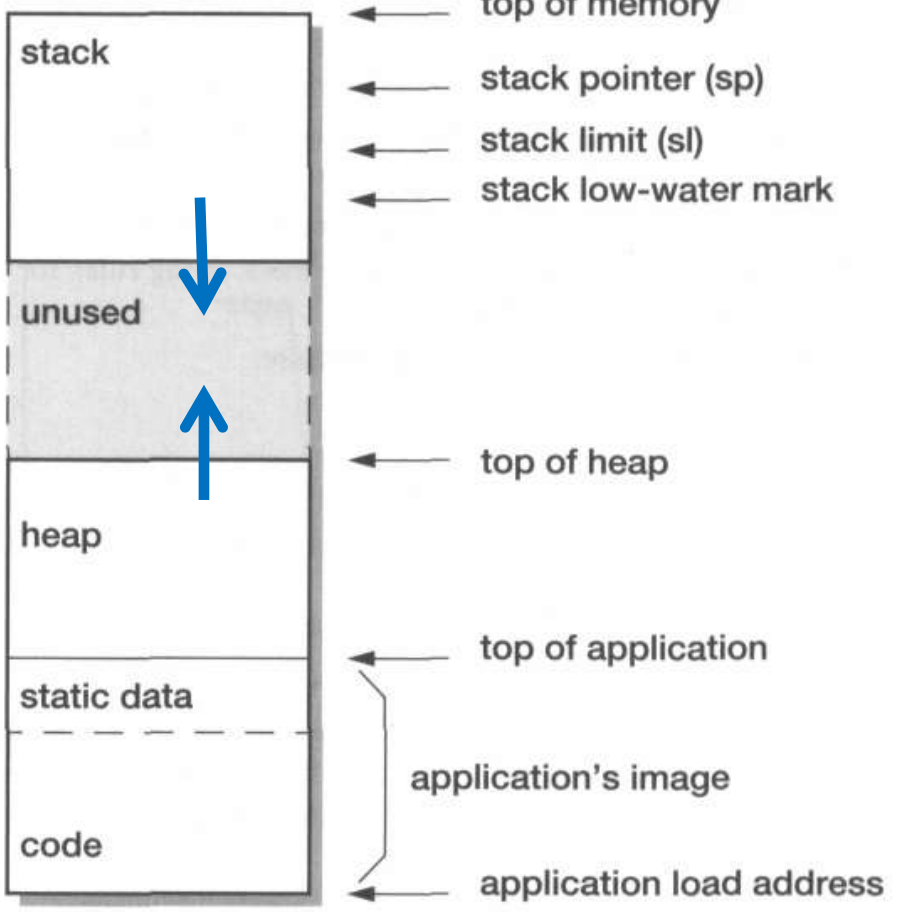
Ref: Ref4_ARM-Cortex-M0+ Devices Generic User Guide, Section 2.1.3 Core Registers

Standard ARM (32 bit) Address Space Model

Upper Address (0xFFFF FFFF)

32 Bit Physical Address Space of size **4 GB**

Lower Address (0x0000 0000)



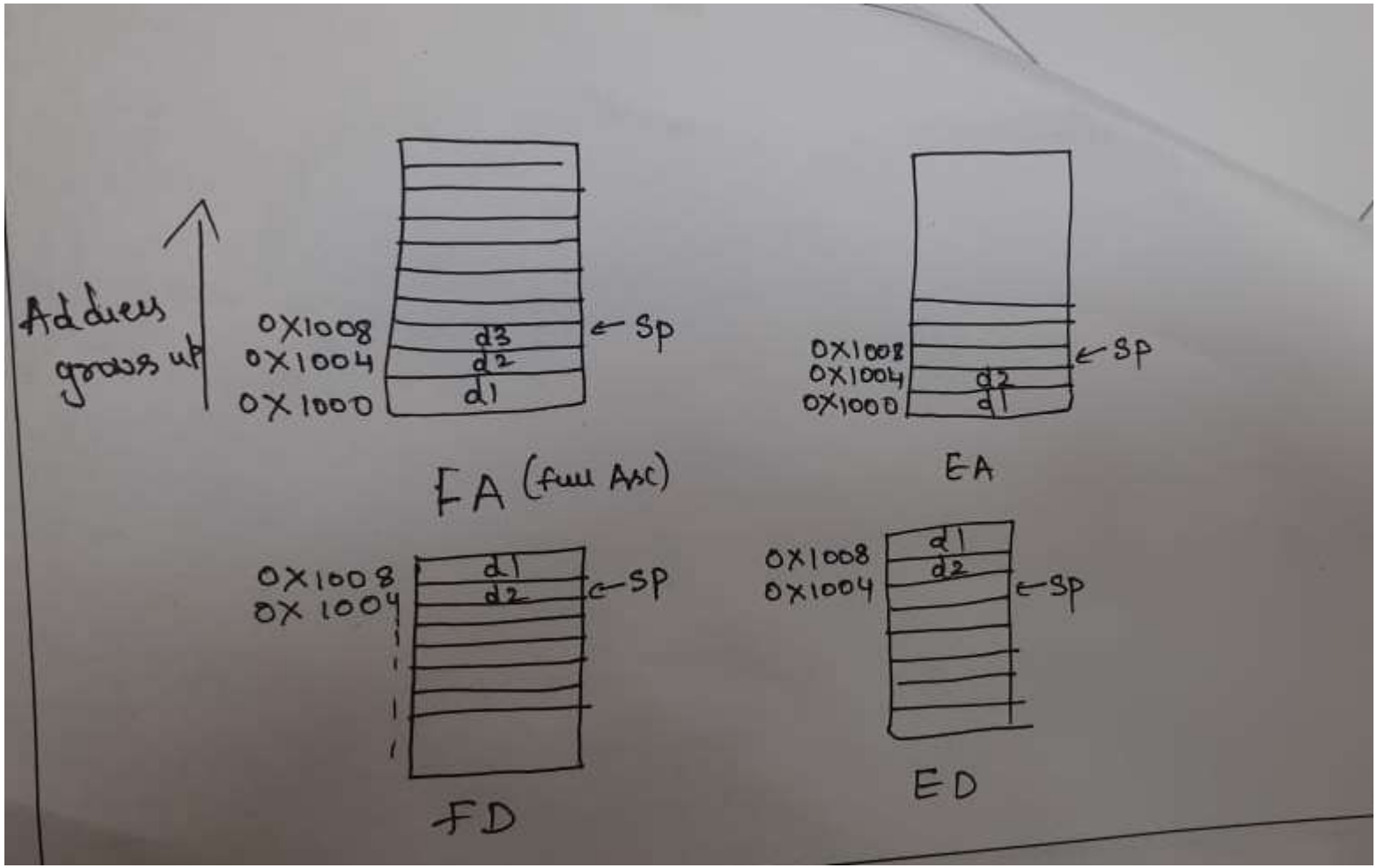
Stack Addressing modes

Stack Processing

ARM support for all four forms of stacks

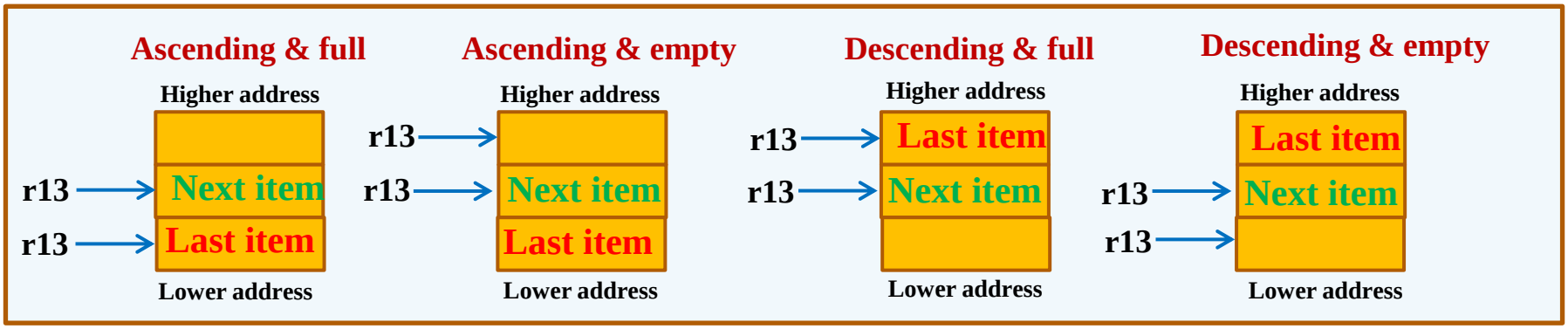
- **Full ascending (FA):** grows up; stack pointer points to the highest address containing a valid data item
- **Empty ascending (EA)**: grows up; stack pointer points to the first empty location
- **Full descending (FD):** grows down; stack pointer points to the lowest address containing a valid data item
- **Empty descending (ED):** grows down; stack pointer points to the first empty location below the stack

Stack Addressing modes



Various Stack Addressing Modes

- The address to be used to store a data value in the stack, is not known at the time the program is compiled or assembled
- A stack is usually implemented as a linear memory space which
  - Grows up (an **ascending** stack) or
  - Grows down (a **descending** stack) in memory as data is pushed into it
- Stack shrinks back as data is removed or popped out
- **Stack Pointer (r13)** holds the address of the current top of the stack
  - Either by pointing to the last valid data item pushed onto the stack (a **full** stack) or
  - By pointing to the vacant slot where the next data item will be placed (an **empty** stack)

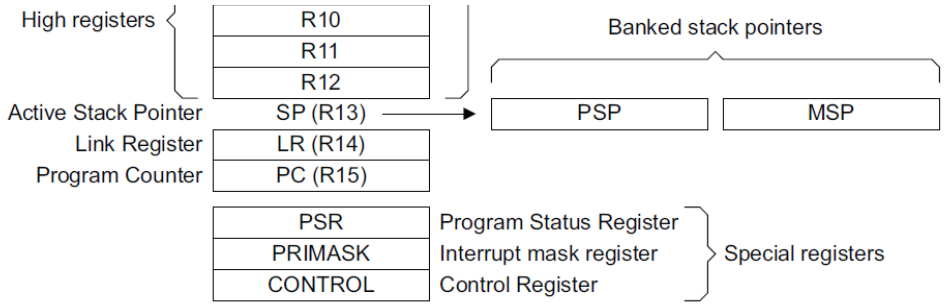


Stack Implementation in ARM

- The choice of implementing one of the **four types** of stack in ARM is with the system designer
  - HW decides the particular type of stack implementation
- A particular type of stack implementation needs to be followed for the application to work together
- There are different instructions available in ARM to implement any of the four types of stacks
- Different register transfer instructions are supported to implement these four types of stack by advanced families of ARM (not in Cortex-M0+)
- **Arm Cortex-M0+** only supports **full-descending mode** and it is not a configurable option here.
- There also **PUSH** and **POP** instructions available in ARM which always assume a **full descending** stack
  - This matches with the convention of stack space growing towards the lower address, towards the heap space
- The width of the stack is 4 bytes (32 bits) always. For every push operation the stack pointer is decremented by 4 before a new value is written into the stack

Arm-Cortex-M0+: Stack Implementation

- The processor uses a **full descending stack**.
  - This means the stack pointer indicates the last stacked item on the stack memory.



  - When the processor pushes a new item onto the stack, it decrements the stack pointer and then writes the item into the new memory location.
- The processor implements two (banked) stacks, the **main stack (MSP)** and the **process stack (PSP)**, with independent copies of the stack pointer.
  - The processor can either be in **Thread** or **Handler mode**.
- In Thread mode, the CONTROL register controls whether the processor uses the MSP or PSP. In Handler mode, the processor always uses the MSP.

Processor mode	Used to execute	Optional privilege level for software execution	Stack used
Thread	Applications	Privileged or unprivileged[a]	Main stack or process stack[a].
Handler	Exception handlers	Always privileged	Main stack.

a. See *CONTROL register* on page 2-8.

Note: Different modes will be covered later.

Ref:Ref4_ARM-Cortex-M0+ Devices Generic User Guide, Section 2.1.2 Stacks

Arithmetic Condition Codes

Arithmetic Conditional Flags

- All the processors have the following conditional flags to indicate the result of Integer Arithmetic operations in the ALU (Arithmetic Logic Unit)
  - Sign Flag (**N**)
  - Zero Flag (**Z**)
  - Carry Flag (**C**)
  - Overflow Flag (**V**)
- In the following sections let us understand the significance of each of them

Definitions of Arithmetic Condition Codes

- **Sign Flag (N): Negative**
  - The last ALU operation which changed the flags produced a negative result (the **Most Significant Bit** of the 32-bit result was a **one**)
- **Zero Flag (Z)**
  - The last ALU operation which changed the flags produced a zero result (every bit of the 32-bit result was zero)
- **Carry Flag (C)**
  - The last ALU operation which changed the flags generated a carry-out, either as a result of an arithmetic operation in the ALU or from the shifter
- **oVerflow Flag (V)**
  - The last arithmetic ALU operation which changed the flags generated a result which cannot be represented within the range of signed values

Interpretation of NZCV Flags

The numbers added in the previous quiz were:

1101

1011

	Binary Values	Unsigned Value	Signed Value	Flags for Unsigned ZC	Flags for Signed NZV
	1101	13	-3		
+	1011	11	-5		
Result	1000	8	-8	01	100

Wrong

Correct

Note: The Overflow (**V**) flag is set to indicate whether the **signed result** after the operation can be fit into the available bits or not. Here **V** is cleared because **-8** can be accommodated within four bits.

Quiz 2

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

Choose the correct option:

1. Consider a **four bit ALU** which does four bit arithmetic. When the following four bit numbers are added what is the status of **NZCV** flags after the addition is performed?

   1101
   + 1011

a) NZCV = 0111
b) NZCV = 1000
c) NZCV = 1001
d) NZCV = 1010

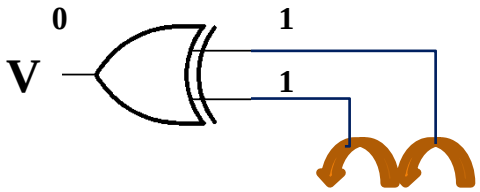
Correct option: d

Note: Relevant flags needs to be interpreted after the above addition based on whether a signed or unsigned result was needed. Let us spend some time on this.

About the Overflow Flag

How is Overflow (V) Flag Set?

Note: Overflow (**V**) is got by **XOR**ing the carry coming out of the bits **prior to Most Significant Bits** and the carry from the **Most Significant Bits**.



Carry Flag:	**1**	**1**	**1**	**0**	**1**
		1	**0**	**1**	**1**
Result of addition:		**1**	**0**	**0**	**0**

One more Example

Let us take another example:

0111

+ 0011

	Binary Values	Unsigned Value	Signed Value	Flags for Unsigned ZC	Flags for Signed NZV
	0111	7	7		
+	0011	3	3		
Result	1010	**10**	**-6**	**00**	**101**

Correct **Wrong**

Notes:

1. The **V** (overflow flag) convey whether a **signed** arithmetic has given out a result which is **incorrect**.
2. If numbers are interpreted as **unsigned**, the **overflow flag is irrelevant**.
3. But if the numbers added are interpreted as **signed**, if **V** is set, it means, either two large positive numbers were added and the result became negative or two large negative numbers were added and the result became positive.

Unsigned and Signed Integers

Unsigned and Signed Integers

- In C, an integer variable can be defined either as s**igned** or **unsigned**
- When these variables are declared, compiler allocates memory space either in **data** or **stack** area depending on whether they are **global/static/local** variables
- When any arithmetic operation needs to be performed on these variables, they are brought into registers from memory
- The processor does not have any knowledge on whether a signed or unsigned integer is moved into a register
- When an arithmetic operation is performed they affect all the condition flags **NZCV**
- Later proper instructions are to be used by the compiler to check either **C** flag or **V** flag to interpret the result based on whether the variables are declared as **signed** or **unsigned**.

Assembly Syntax

Calling an asm function from C

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

prog_lec_2_2_2.ino | diagram.json | main_asm.S | Library Manager

```
13  #include<stdio.h>
14  #include<stdlib.h>
15
16  // Uncomment this if when you copy this code to run it on HW
17  //#define RUN_ON_HW
18
19  #ifdef RUN_ON_HW
20  #define Serial1 Serial
21  #endif
22
23  void myPrint(char* fnName, int result);
24
25  // Define external assembly function
26  extern "C" {
27    unsigned int my_asm_fn(void);
28  }
```

```
7   @ Ref: ARMv6M Technical reference to learn about the
8   @ Syntax and usage of asm instructions
9   // Label of the asm function
10  .global my_asm_fn   @ To provide the address of this la
11
12  .text
13  @ *******
14  // This function just returns  value of 200 through r0
15  my_asm_fn:
16   mov r0, #200    @ Return a value of 200 in decimal back
17   bx lr           @ Return from the asm function back to
18
19  .end
20
21  @ End of text or code segment
```

This is part of the program being demonstrated in this lecture:
prog_lec_2_2_2.ino

.S is the extension for the assembly program file.

Assembly Syntax: main_asm.S

- Comments: Starting a line with **@** symbol or double slash //
- **.text** is the starting of the code segment, where assembly instructions are stored
- Labels end with a **colon :**
- **.end** marks the end of a section
- Immediate constants start with a #
- *mov r0, #200*
- Moves the value 200 in decimal into the register **r0**
- bx lr
- **Bx: Branch with exchange**
- **LR** is the link register (**R14**) which has the code address where the function should return back

```
7   @ Ref: ARMv6M Technical reference to learn about the
8   @ Syntax and usage of asm instructions
9   // Label of the asm function
10  .global my_asm_fn   @ To provide the address of this la
11
12  .text
13  @ *******
14  // This function just returns  value of 200 through r0
15  my_asm_fn:
16   mov r0, #200   @ Return a value of 200 in decimal back
17   bx lr          @ Return from the asm function back to
18
19  .end
20
21  @ End of text or code segment
```

This is part of the program being demonstrated in this lecture: **prog_lec_2_2_2.ino**

.S is the extension for the assembly program file.

Technical Reference Manual of Cortex-M0+ (Ref 3)

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

3.3 Instruction set summary

The processor implements the ARMv6-M Thumb instruction set, including a number of 32-bit instructions that use Thumb-2 technology. The ARMv6-M instruction set comprises:

- All of the 16-bit Thumb instructions from ARMv7-M excluding `CBZ`, `CBNZ` and `IT`.
- The 32-bit Thumb instructions `BL`, `DMB`, `DSB`, `ISB`, `MRS` and `MSR`.

Table 3-1 shows the Cortex-M0+ instructions and their cycle counts. The cycle counts are based on a system with zero wait-states.

Table 3-1 Cortex-M0+ instruction summary

Operation	Description	Assembler	Cycles
Move	8-bit immediate	`MOVS Rd, #<imm>`	1
	Lo to Lo	`MOVS Rd, Rm`	1
	Any to Any	`MOV Rd, Rm`	1
	Any to PC	`MOV PC, Rm`	2
Add	3-bit immediate	`ADDS Rd, Rn, #<imm>`	1
	All registers Lo	`ADDS Rd, Rn, Rm`	1
	Any to Any	`ADD Rd, Rd, Rm`	1
	Any to PC	`ADD PC, PC, Rm`	2
	8-bit immediate	`ADDS Rd, Rd, #<imm>`	1
	With carry	`ADCS Rd, Rd, Rm`	1
	Immediate to SP	`ADD SP, SP, #<imm>`	1

Arithmetic Instructions

Syntax: <instruction>{<cond>}{S} Rd, Rn, N

ADC	add two 32-bit values and carry	$Rd = Rn + N +$ carry
ADD	add two 32-bit values	$Rd = Rn + N$
RSB	reverse subtract of two 32-bit values	$Rd = N - Rn$
RSC	reverse subtract with carry of two 32-bit values	$Rd = N - Rn -$!(carry flag)
SBC	subtract with carry of two 32-bit values	$Rd = Rn - N -$!(carry flag)
SUB	subtract two 32-bit values	$Rd = Rn - N$

Problems

```
PRE     r0 = 0x00000000
        r1 = 0x00000002
        r2 = 0x00000001

        SUB r0, r1, r2

POST    r0 = 0x00000001
```

```
PRE     r0 = 0x00000000
        r1 = 0x00000077

        RSB r0, r1, #0      ; Rd = 0x0 - r1

POST    r0 = -r1 = 0xffffff89
```

Problems

EXAMPLE 3.4 This simple subtract instruction subtracts a value stored in register *r2* from a value stored in register *r1*. The result is stored in register *r0*.

```
PRE   r0 = 0x00000000
      r1 = 0x00000002
      r2 = 0x00000001

      SUB r0, r1, r2

POST  r0 = 0x00000001
```

EXAMPLE 3.6 The SUBS instruction is useful for decrementing loop counters. In this example we subtract the immediate value one from the value one stored in register *r1*. The result value zero is written to register *r1*. The *cpsr* is updated with the *ZC* flags being set.

```
PRE   cpsr = nzcvqiFt_USER
      r1 = 0x00000001

      SUBS r1, r1, #1

POST  cpsr = nZCvqiFt_USER
      r1 = 0x00000000
```

Problems

EXAMPLE 3.34 This example shows an ADD instruction with the EQ condition appended. This instruction will only be executed when the zero flag in the *cpsr* is set to 1.

```
; r0 = r1 + r2 if zero flag is set
ADDEQ r0, r1, r2
```

Only comparison instructions and data processing instructions with the S suffix appended to the mnemonic update the condition flags in the *cpsr*.

EXAMPLE 3.35 To help illustrate the advantage of conditional execution, we will take the simple C code fragment shown in this example and compare the assembler output using nonconditional and conditional instructions.

```
while (a!=b)
{
  if (a>b) a -= b; else b -= a;
}
```

Demo of Asm Program

prog_lec_2_2_2.ino and main_asm.S

• Program

```
AREA AddReg,CODE,READONLY ;Name this block of
code.
  ADR r0,ThumbProg +1 ;Generate branch target address
 ;and set bit 0,hence arrive
 ;at target in Thumb state.
 BX r0 ;Branch exchange to ThumbProg.
 CODE16 ;Subsequent instructions are Thumb code.
ThumbProg
 MOV r2,#2 ;Load r2 with value 2.
 MOV r3,#3 ;Load r3 with value 3.
 ADD r2,r3 ;r2 =r2 +r3
 ADR r0,ARMProg
 BX r0
 CODE32 ;Subsequent instructions are ARM code.
ARMProg
 MOV r4,#4
 MOV r5,#5
 ADD r4,r4,r5
 END
```